# SMOOTHIE: (Multi-)Scalar Multiplication Optimizations On TFHE

Xander Pottier, Jan-Pieter D'Anvers, Thomas de Ruijter, Ingrid Verbauwhede

COSIC KU Leuven, Leuven, Belgium, {firstname.lastname}@esat.kuleuven.be

**Abstract.** The (Multi-)Scalar multiplication is a crucial operation during FHE-related AI applications, and its performance has a significant impact on the overall efficiency of these applications. In this paper we introduce SMOOTHIE: (Multi-)Scalar Multiplication Optimizations On TFHE, introducing new techniques to improve the performance of single- and multi-scalar multiplications in TFHE. We show that by taking the bucket method, known from the Elliptic Curve field, significant improvements can be made. However, as the characteristics between TFHE and Elliptic Curves differ, we need to adapt this method and introduce novel optimizations. We propose a new negation with offset technique that eliminates direct carry propagation after ciphertext negation. Additionally, we introduce powershift aggregation and bucket merging techniques for the bucket aggregation step, which exploit TFHE properties to substantially reduce bootstrap operations. Specifically, in the multi-scalar multiplication case, we implement a bucket doubling method that eliminates the need for precomputation on each input ciphertext. Our implementation is integrated in the TFHE-rs library and achieves up to $\times 2.05$ speedup for single-scalar multiplication compared to the current state-of-the-art, with multi-scalar multiplication improvements up to $\times 7.51$, depending on the problem size.

**Keywords:** Scalar Multiplication · Multi-Scalar Multiplication · Torus Fully Homomorphic Encryption · Bucket Method

## 1 Introduction

Machine Learning and the widespread use of AI in our daily lives grew significantly in recent years. However, these AI models are often hosted on cloud servers. Although standard encryption techniques can securely transfer and store data in the cloud, it still remains vulnerable during processing as it must be decrypted on the server, which raises privacy concerns.

Fully Homomorphic Encryption (FHE) offers a solution to this privacy risk. It enables computations on sensitive data without revealing the contents to the server. With FHE, we can encrypt our data and send it to any external server for computation, ensuring that the data stay secure throughout the process.

However, practical FHE applications are often slow due to an operation called bootstrapping. FHE encryptions are secure by introducing a small noise term. If the noise grows excessively, the result becomes incorrect. To prevent this, we need to perform bootstrapping operations to reduce the noise.

Over the years, multiple FHE schemes have been developed, each with its own set of advantages and disadvantages. For example, second-generation schemes like BFV [FV12], BGV [BGV12], and CKKS [CKKS17], are designed to perform SIMD operations efficiently, but their bootstrap operation is relatively slow. On the other hand, Torus Fully Homomorphic Encryption (TFHE) [CGGI18] is a popular third-generation FHE scheme that does not have

the SIMD property, but can perform fast bootstrapping on small integers. Moreover, TFHE bootstrapping is also unique as it can evaluate any function, while also reducing the noise, known as a programmable bootstrap (PBS). In this paper, we will focus on TFHE.

AI applications typically involve two types of operations: multiply-accumulate and a non-linear activation function. In this paper, we will focus on the multiply-accumulate, which involves multiplying multiple inputs with the kernel weights of the AI model and then adding the results together. When implementing these applications using FHE, the kernel weights are usually kept in plaintext, while the input from the client is encrypted using FHE. This means that all linear operations can be expressed as a multiplication between multiple input TFHE ciphertexts ($ct_i$) and multiple plaintext scalars ($s_i$), also known as a multi-scalar multiplication:

$$\sum_{i=1}^{k} s_i \cdot ct_i.$$

In this paper, our objective is to reduce the latency of both single- and multi-scalar multiplication problems.

**Related Work** There are only few previous works that focus on accelerating scalar multiplication in TFHE, despite its relevance in numerous AI applications. One work by Klemsa et al. (Parmesan) [KO23] uses an addition/subtraction chain method derived from the elliptic curve domain to perform scalar multiplication. This method reduces the total number of required additions, but these additions are highly sequential, significantly impacting overall performance. Moreover, the most efficient addition/subtraction chain for a specific scalar varies widely, leading to highly variable results depending on the scalar. The scalar multiplication implementation by Zama in the TFHE-rs library [Zam22] adopts a schoolbook multiplication approach. This allows the use of efficient vector addition functions that parallelize the bootstrap operations, resulting in improved latency. This approach will be explained in detail in section 3.1.

**Our Contributions**

In this paper, we present an optimized approach for both single-scalar and multi-scalar multiplication operations within TFHE. Our method draws inspiration from Pippenger's bucket method [Pip76], commonly used in the Elliptic Curve domain. While the bucket method was originally proposed for elliptic curve multi-scalar multiplication problems, we show that by leveraging the unique features of TFHE, it is also beneficial in the single-scalar multiplication case.

Furthermore, we introduce a technique called "negation with offset" to minimize overhead when multiple negatives of a ciphertext have to be calculated. This method effectively eliminates any potential underflow that can occur when calculating the negative of a ciphertext, thereby eliminating the need for direct carry propagation on each ciphertext.

Next, we propose a "bucket merging" technique that reduces the number of bootstrap operations in the bucket aggregation step by taking advantage of the unique features of TFHE.

Finally, for the multi-scalar multiplication case, we introduce a "bucket doubling" technique that eliminates the need to precompute $2 \cdot ct_i$ for each input ciphertext $ct_i$, significantly reducing the required number of bootstrap operations for larger multi-scalar multiplications.

Tables 1 and 2 provide an overview of the asymptotic complexities, expressed in Big O notation, for the optimizations discussed in this paper for single- and multi-scalar multiplications in TFHE, respectively. All the code is open-source and can be found on our Github repository.[1]

[1] https://github.com/KULeuven-COSIC/SMOOTHIE

Table 1: Comparison of the asymptotic number of ciphertext bootstrap (CTB) operations for each of the proposed optimizations for a single-scalar multiplication of the scalar $s$ with a TFHE ciphertext. $c$ indicates the window size.

| | Precomp | Accumulation | Aggregation |
|---|---|---|---|
| TFHE-rs [Zam22] | 1 | $\mathcal{O}\left(\frac{\log_2(s)}{2}\right)$ | 0 |
| TFHE Pippenger | 1 | $\mathcal{O}\left(\frac{2^c-1}{2^c} \cdot \frac{\log_2(s)}{c}\right)$ | $\mathcal{O}\left(\frac{5}{8}2^{2c}\right)$ |
| Sliding Windows | 1 | $\mathcal{O}\left(\frac{\log_2(s)}{c+1}\right)$ | $\mathcal{O}\left(\frac{5}{16}2^{2c}\right)$ |
| Negative Windows | 1 | | $\mathcal{O}\left(\frac{5}{64}2^{2c}\right)$ |
| Powershift Aggregation | 1 | | $\mathcal{O}\left(\frac{25}{16}2^{c}\right)$ |
| Bucket Merging | 1 | | $\mathcal{O}\left(\frac{5}{8}2^{c}\right)$ |

Table 2: Comparison of the asymptotic number of ciphertext bootstrap (CTB) operations for each of the proposed optimizations for a multi-scalar multiplication of $k$ scalars and TFHE ciphertexts. $c$ indicates the window size.

| | Precomp | Accumulation | Aggregation |
|---|---|---|---|
| TFHE-rs [Zam22] | $k$ | $\mathcal{O}\left(\begin{array}{c} k \cdot \frac{\log_2(s)}{2} + \\ k \cdot \log_{10}(\log_2(s)) \end{array}\right)$ | $\mathcal{O}\left(\frac{5}{8}k\right)$ |
| Prevent Carry Propagate | $k$ | $\mathcal{O}\left(k \cdot \frac{\log_2(s)}{2}\right)$ | $\mathcal{O}\left(\frac{10}{8}k\right)$ |
| TFHE Pippenger | $k$ | $\mathcal{O}\left(k \cdot \frac{\log_2(s)}{c+1}\right)$ | $\mathcal{O}\left(\frac{5}{8}2^{c}\right)$ |
| Bucket Doubling | 0 | $\mathcal{O}\left(k \cdot \frac{\log_2(s)}{c+1}\right)$ | $\mathcal{O}\left(\frac{18}{8}2^{c}\right)$ |

# 2 Preliminaries

In this section, we will explain how integers are represented using multiple TFHE ciphertexts. Following this, we will introduce several fundamental operations that will be utilized throughout the paper.

## 2.1 Integers in TFHE

As mentioned in the previous section, a single TFHE ciphertext can only encrypt small integers. For instance, one of the most commonly used parameter sets in applications enables the encryption of 2-bit messages in a single ciphertext. For larger messages, we divide the message

$m$ into groups of two bits, denoted by $m_i$, and encrypt each of them separately, as follows:

$$Enc(m) = [ct_{\mathcal{B}}, ..., ct_2, ct_1]$$

$$\text{where} \quad ct_i = Enc(m_i) \quad \text{and} \quad m = \sum_{i=1}^{\mathcal{B}} m_i 2^{2(i-1)} \quad \text{and} \quad \mathcal{B} = \lceil \log_2(m)/2 \rceil.$$

Each of these ciphertexts is referred to as a "block", indicating that a message $m$ can be represented using $\mathcal{B} = \lceil \log_2(m)/2 \rceil$ blocks. To retrieve the correct integer after decryption we have to decrypt each block separately after which we add the result together as follows:

$$m = Dec(Enc(m)) = \sum_{i=1}^{\mathcal{B}} Dec(Enc(m_i)) \cdot 2^{2(i-1)}$$

To perform computations on these blocks, there is also allocated space for potential carry bits. These carry bits are always zero after encryption and the number of carry bits are often equal to the number of message bits. As a result, the encryption of a message $m$ can be represented as follows:

$$Enc(m) = \boxed{0\,0 \mid m_{\mathcal{B}}} \cdots \boxed{0\,0 \mid m_2} \; \boxed{0\,0 \mid m_1}$$

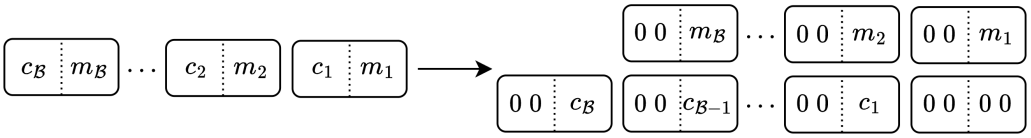
In the 2-bit message and 2-bit carry case, the possible values $m_i$ that can occur in the message space of each block range from 0 to 3, while the maximum value that can be stored in the blocks using the carry bits is 15. To add ciphertexts together, we add the corresponding blocks using a homomorphic addition. When adding two new ciphertexts together, the result is a new ciphertext with the same number of blocks, where the carry bits are at most one (since $(11)_2 + (11)_2 = (110)_2$). During operations, it is essential to closely monitor the maximum carry value to prevent overflow, which could lead to incorrect results. To address this issue and enable continuous operations, we use the message and carry extraction (MC) operation, which will be detailed in the next section.

## 2.2 Message and Carry Extraction

When the carry bits of a ciphertext are completely filled, we have to perform a Message and Carry extraction (MC) operation. This operation extracts both carry and message bits for each block in the ciphertext using two separate programmable bootstrap operations

$$\text{MC}(ct_i) = \begin{cases} m_i & = \text{PBS}(ct_i \bmod 4) \\ c_i & = \text{PBS}(\lfloor ct_i/4 \rfloor) \end{cases}$$

This results in two new ciphertexts where the carry bits are empty for each ciphertext:




Note that the resulting carry ciphertext is shifted by one position, ensuring that the carries align appropriately with the message. Afterward, the ciphertext representing the carry bits can be added to the message ciphertext to form a single ciphertext. Note that, the carry bits in the resulting ciphertext are not zero after the addition (since $(11)_2 + (11)_2 = (110)_2$). This reduces the number of additions that can be performed before the carry bits are filled again, requiring a new MC operation. In the next section, we will introduce the carry propagate function, which ensures that the carry bits are empty.

## 2.3 Carry Propagate

Although the previous MC operation partially clears the carry bits, sometimes it is necessary to reset them completely to zero. This can be achieved using the Carry Propagate function. As the name implies, the carry propagate operation sequentially propagates the carry block by block throughout the entire integer. If the carry bits of the initial ciphertext are fully occupied, an MC operation must be performed first to prevent overflow when an additional carry is added to the ciphertext block. After this, the carry propagation can commence by extracting the message and carry from the least significant block and adding the carry to the next block. This process continues with each subsequent block performing the same operation. Figure 1 shows this operation for a ciphertext consisting of three blocks, where the resulting ciphertext blocks with empty carry bits are indicated in green.

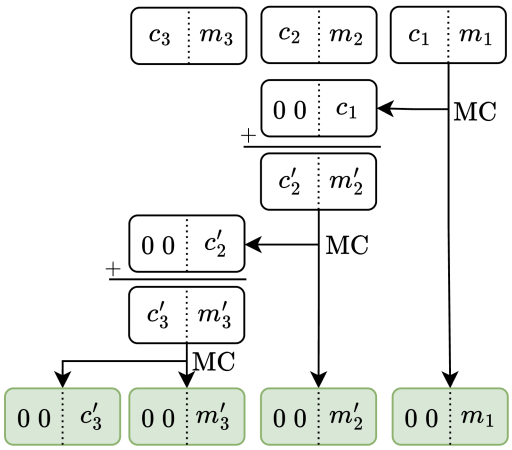



Figure 1: Visualization of the Carry Propagate function for a ciphertext consisting of three blocks. The resulting ciphertext blocks are indicated in green. Note that the carry bits may not be completely filled at the start of the operation.

The carry propagation function requires in total $2\mathcal{B}$ PBS operations, with $\mathcal{B}$ the total number of ciphertext blocks. Due to the sequential operations and the fact that PBS operations are the main bottleneck in TFHE, the algorithm is relatively slow. To accelerate this process for large integers, optimizations exist that involve more PBS operations, in the order of $\mathcal{O}(\mathcal{B}\log_2(\mathcal{B}))$, but allow for greater parallelism, improving latency. More details on this parallel implementation can be found in the Zama TFHE-rs handbook [BdBB+25].

## 2.4 Vector Addition

The last fundamental operation we will introduce is the Vector Addition. This operation takes a vector of $N$ ciphertexts, each containing $\mathcal{B}$ blocks with carry bits zero, as input. Its goal is to efficiently calculate the sum of all $N$ ciphertexts.

The vector addition is done efficiently by first splitting each ciphertext into its corresponding ciphertext blocks. The blocks are then grouped together in a block vector based on their position in the ciphertext. For example, all least significant blocks are grouped in a least significant block vector. This results in $\mathcal{B}$ block vectors. Next, each block vector is processed separately during the operation. Since the carry bits of the initial elements in each vector are empty, up to five elements can be added together before the carry bits become full (as $5\cdot(11)_2=(1111)_2$). At this point, the MC operation is used to extract both the message and carry. The message is added back to the same block vector, while the carry is added to the

adjacent block vector. This process continues until each block vector holds a single ciphertext. Figure 2 shows a simple example of how the vector addition works for a vector of six ciphertexts each consisting of three blocks. The result of the vector addition is indicated in purple.

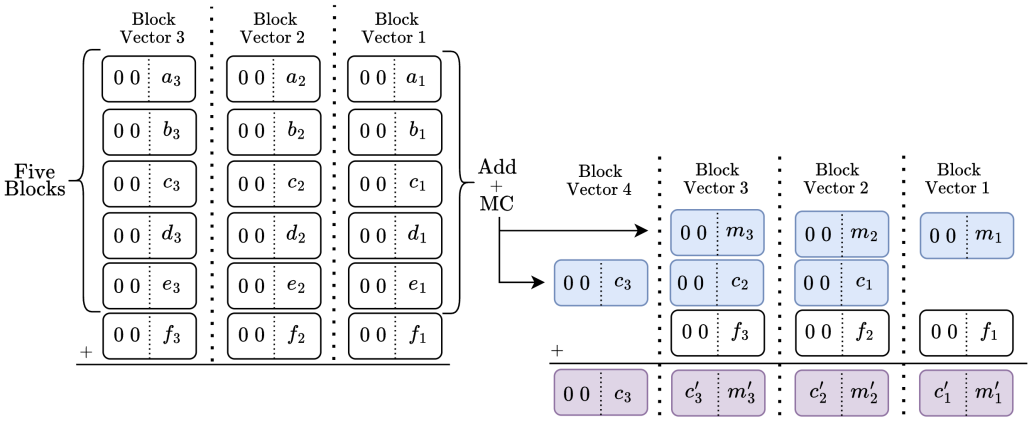



Figure 2: Visualization of the Vector Addition function for a vector of six ciphertexts consisting of 3 blocks each

The total number of PBS operations required for vector addition depends on the number of elements in each block vector. However, not all block vectors have the same number of elements. Some ciphertexts can be represented with fewer blocks, or they can be multiplied by a power of two that makes some blocks zero. These zero blocks are not added to their corresponding block vector. The exact complexity is detailed in the Zama TFHE-rs handbook [BdBB+25].

In this paper, we will assume that each block vector has the same number of elements $N$. Starting from theorem 26 in [BdBB+25], we can derive that the total number of PBS operations is approximately:

$$nb_{PBS} \approx \begin{cases} \frac{5}{8}\mathcal{B}\cdot(N-5) & \text{if } N>5 \\ 0 & \text{otherwise} \end{cases}$$

This shows that given a vector of ciphertexts of $\mathcal{B}$ blocks, the number of PBS operations for the vector addition is linearly dependent on the total number of elements in the block vectors. For simplicity, in this paper, we will measure the cost of the proposed optimizations as the time to execute a programmable bootstrap on a single encrypted integer consisting of $\mathcal{B}$ blocks. We refer to these operations as ciphertext bootstrap (CTB) operations. Therefore, one CTB operation is equivalent to $\mathcal{B}$ PBS operations. To maintain a clear notation throughout the paper, we also define the function $\mathcal{V}_{\square}$ as follows:

$$\mathcal{V}_{\square}(x) = \begin{cases} \frac{5}{8}(x-5) & \text{if } x>5 \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

This function defines the number of CTB operations required for vector addition of $x$ ciphertexts. The $\square$ symbol denotes that each block vector in the addition contains an equal number of elements, thus representing the vector addition as a "rectangle".

## 3 Single-Scalar Multiplication

The single-scalar multiplication involves multiplying a plaintext scalar with a ciphertext and is a fundamental operation in many applications, including neural networks. Due to the

frequent use, minimizing latency is crucial. In this section, we introduce our improved scalar multiplication technique in TFHE, inspired by methods from the cryptographic domain of Elliptic Curves (EC). We adapt this technique to leverage the unique features of TFHE and introduce additional improvements. We start this section by explaining the current approach for the scalar multiplication used in typical TFHE applications and then demonstrate how our technique improves upon this. Given that CTB operations are the primary bottleneck in TFHE, our focus will be on reducing these operations as much as possible.

## 3.1 Current approach

The current implementation [Zam22] of the single-scalar multiplication follows a method similar to the well-known schoolbook multiplication algorithm. This method processes the scalar bit by bit. When a bit is one, a shifted version of the ciphertext is added to a vector. Afterward, all elements of the vector are summed to obtain the final result.

Given that we are using the TFHE parameter set with a 2-bit message and 2-bit carry, we can perform a multiplication by powers of four (known as a blockshift) at no cost. Multiplication by two requires one PBS operation per block of the ciphertext, which is equivalent to one CTB operation. This multiplication by two is performed once on the ciphertext at the start.

In the current approach, we loop over the bits in the scalar. If a bit at an even index is one, the original $ct$ is blockshifted and added to a vector. If a bit at an odd index is one, we use the precomputed multiple of two, $2 \cdot ct$, instead. After processing each bit of the scalar, we have a vector of shifted ciphertexts, which are then efficiently combined using the vector addition function. An example of this schoolbook method scalar multiplication for a scalar of 5-bits and a ciphertext consisting of four blocks is given in figure 3.

Note that we often work with a power of two modulus in TFHE, for example $2^8$. If we use this modulus for the example in figure 3, we only have to calculate the green colored parts of the multiplication as the other parts are automatically reduced to zero.

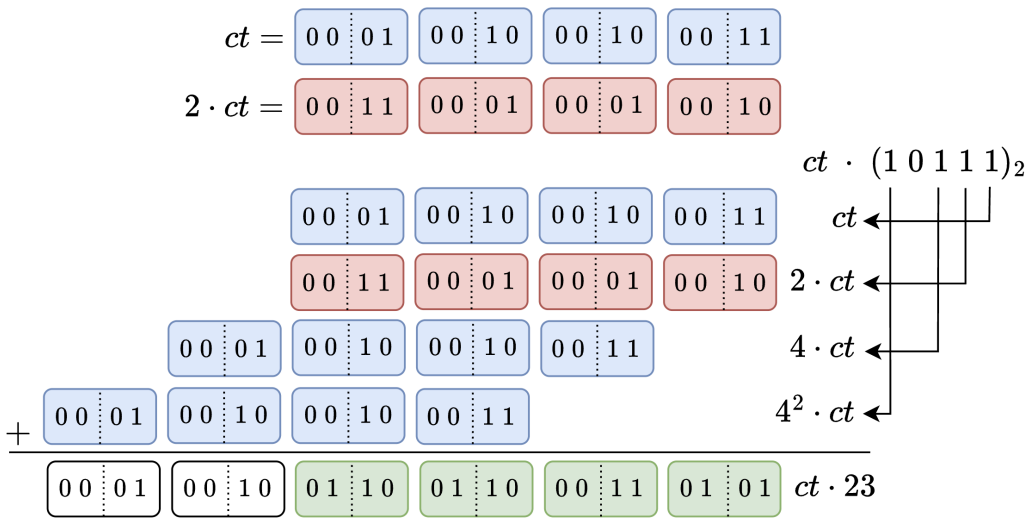



Figure 3: Current approach for the scalar multiplication in TFHE. When using $2^8$ as modulus only the green blocks have to be calculated.

With this current approach, the total number of times the ciphertext is added to the vector corresponds to the number of one bits in the scalar $s$. On average, this is half the binary

length of the scalar $s$:

$$N = \log_2(s)/2.$$

Starting from equation 1, we can calculate the required number of CTB operations using the vector add. However, equation 1 assumed that each block vector had the same number of elements. This is not the case here, only the most significant block has $N$ elements, while it gradually reduces going to the least significant block. We can approximate the total number of CTB operations as half the number of operations expected if each block vector had the same number of elements $N$. Also, the multiplication by two at the beginning requires one CTB operation. This results in a total of:

$$nb_{CTB} = 1 + \mathcal{V}_{\square}\left(\frac{\log_2(s)}{2}\right)/2.$$

For simplicity in the rest of the paper we define a new function $\mathcal{V}_{\triangle}(x)$ as:

$$\mathcal{V}_{\triangle}(x) = \mathcal{V}_{\square}(x)/2 = \begin{cases} \frac{5}{16}(x-5) & \text{if } x > 5 \\ 0 & \text{otherwise} \end{cases}$$

This function represents the total CTB operations in a vector addition where the number of elements in the block vectors have a maximum of $x$ elements and decrease gradually. As shown in Figure 3, this can also be represented in a triangle structure, hence the use of the $\triangle$ symbol. Using this function, we express the total number of CTB operations in the current approach of the scalar multiplication as:

$$nb_{CTB} = 1 + \mathcal{V}_{\triangle}\left(\frac{\log_2(s)}{2}\right).$$

To improve the latency of scalar multiplication, we aim to reduce the total number of elements in the vector as much as possible without introducing significant overhead. In the following sections, we will discuss a promising technique from other fields of cryptography and apply it to TFHE.

## 3.2 Window Scalar Multiplication

As discussed in the previous section, the primary bottleneck in the current scalar multiplication approach is the total number of elements in the vector. We will now demonstrate that it is possible to significantly reduce the total number of elements with a method that resembles the Pippenger or Bucket method known from Elliptic Curve Cryptography (ECC) [Pip76]. This method is commonly used in multi-scalar multiplication scenarios within ECC but has never been applied to FHE. In this section, we will first provide a short introduction to the Pippenger method. Afterward, we will illustrate how the TFHE case differs from the ECC one and how we can adapt the bucket method to also improve on the current approach in the single-scalar multiplication case.

### 3.2.1 Elliptic Curve Pippenger Method

In ECC, one of the most common operations is the multiplication of multiple scalars, $s_k$, with elliptic curve points, $P_k$. Since there is no direct way to multiply a scalar with an elliptic curve point, this operation must be performed using only elliptic curve additions. Pippenger introduced a method that significantly reduces the total number of additions, known as the bucket method [Pip76]. Generally, the bucket method divides scalar multiplication into two steps: the Bucket Accumulation and the Bucket Aggregation step.

In the Bucket Accumulation step, each scalar $s_k$ is divided into subscalars $s_{k,w}$ of $c$ bits, resulting in a total of $W = \lceil \log_2(s)/c \rceil$ subscalars. Every subscalar is associated with a

window $w$ that is made up of a set of $2^c-1$ buckets (essentially storage elements) representing the possible values of the subscalars $s_{k,w}$. Then, each point $P_k$ is added to a bucket for every window $w$, corresponding to the value of $s_{k,w}$. For example, the Bucket Accumulation step for a multi-scalar multiplication of $N$ scalars and EC points works as follows:

$$\text{Bucket Accumulation} = \begin{cases} s_k = \sum_{w=0}^{W-1} 2^{cw} s_{k,w} \\ \\ \forall k \in [0, N-1] \\ \forall w \in [0, W-1] \end{cases} \quad B_{w,s_{k,w}} \mathrel{+}= P_k$$

Where $B_{w,s_{k,w}}$ represents the bucket in window $w$ with index $s_{k,w}$.

Once all the subscalars are processed, the bucket aggregation step begins. In this step, all buckets within one window are multiplied by their corresponding scalar (implemented as a series of additions). Afterward, all windows are added together to get the result $S$:

$$\text{Bucket Aggregation} = \begin{cases} B_w = \sum_{i=1}^{2^c-1} i B_{w,i} \quad \text{with} \quad iB_{w,i} = \sum_{l=0}^{i-1} B_{w,i} \\ \text{S} = \sum_{w=0}^{W-1} 2^{cw} B_w \end{cases}$$

This algorithm essentially delays the multiplication by the subscalar value until the very end, thereby reducing the total number of EC additions. For instance, instead of calculating $9P_1 + 9P_2$, we first perform the addition in $B_{0,9}$ and then multiply the result by 9: $9 \cdot (P_1 + P_2)$. This is implemented as a series of EC additions since a multiplication cannot be performed directly.

Besides the original Pippenger method, there are multiple optimizations that further reduce the total number of additions. For instance, one such optimization is called the BGMW or Precomputation method [BGMW92]. In ECC, elliptic curve points are often constant, allowing us to precompute them and store the results in memory for efficient scalar multiplication. The BGMW method precomputes specific power of two multiples of the points, such that the number of buckets reduces from one set per window to a single set:

$$\text{Bucket Accumulation} = \begin{cases} \forall k \in [0, N-1] \\ \forall w \in [0, W-1] \end{cases} \quad B_{s_{k,w}} \mathrel{+}= P_{k,w} \quad \text{with} \quad P_{k,w} = 2^{cw} P_k$$

This method significantly reduces the total number of windows, thereby reducing the total latency of the bucket aggregation step. However, it requires storage for all multiples of the point $2^{cw} P_k$, which can become substantial for large scalar sizes.

Pippenger demonstrated that this method is optimal for large EC multi-scalar multiplication cases. However, we will show that it also brings improvements in the TFHE single-scalar multiplication by adapting and introducing new optimizations.

### 3.2.2 TFHE adapted Pippenger Method

The TFHE situation is quite similar to the ECC case, where, instead of an elliptic curve point, we have a TFHE-encrypted ciphertext. Multiplying a plaintext scalar with a TFHE ciphertext is not feasible because of potential overflows that can occur, requiring us to use multiple ciphertext additions. However, there are significant differences between ECC and TFHE that enable us to introduce new optimizations and even apply the bucket method to the single-scalar multiplication in TFHE, which was not possible in ECC.

Recall that in ECC the points are known beforehand, which enables us to introduce precomputation techniques, such as the BGMW method, that trade memory storage for efficiency. Moreover, the bucket method is typically not applied for single-scalar multiplication in ECC, as the number of points that are added to the buckets is too small in comparison to the number

of buckets per window. By decreasing the window size $c$ and using the BGMW method, we add more points to every window, but this requires the storage of a lot of precomputated values.

In the TFHE case, the situation is different. The ciphertexts are not known beforehand, which prevents us to directly use any precomputation technique. However, as discussed before, a multiplication with powers of four can be performed for free in TFHE. Moreover, by performing one multiplication by two at the beginning (requiring one CTB operation), we can multiply the ciphertext with any power of two at runtime. This allows us to implement the bucket method with the BGMW optimization for smaller window sizes $c$ without requiring any storage for precomputations.

Applying the bucket BGMW method with a window size $c$ to the TFHE case, we can rewrite a single-scalar multiplication with a TFHE ciphertext as follows:

$$ct \cdot s = ct \cdot \sum_{w=0}^{W-1} 2^{c \cdot w} s_w = \sum_{w=0}^{W-1} s_w \cdot ct_w \quad \text{and} \quad ct_w = 2^{c \cdot w} ct \tag{2}$$

where $ct_w$ can be calculated at negligible cost as it only involves shift operations. This simplifies the bucket method to the following form:

$$ct \cdot s = \underbrace{\sum_{i=0}^{2^c-1} i \cdot B_i}_{\text{Bucket Aggregation}} \quad \text{and} \quad B_i = \underbrace{\sum_{w=0\,:\,s_w==i}^{W-1} ct_w}_{\text{Bucket Accumulation}} \tag{3}$$

In terms of complexity, the total number of elements stored across all buckets is equal to the total number of windows, which is $\log_2(s)/c$. However, the window values of zero can be ignored. Therefore, we end up with:

$$N = \frac{2^c - 1}{2^c} \cdot \log_2(s)/c$$

number of elements in all buckets. We can assume that the number of elements in each bucket is distributed evenly, which implies that there are $2^c - 1$ vector additions with $N/(2^c - 1)$ elements in each addition. Therefore, the total number of CTB operations performed during the bucket accumulation stage is equal to:

$$nb_{CTB,acc} = (2^c - 1) \cdot \mathcal{V}_{\triangle}\left(\frac{N}{2^c - 1}\right) = (2^c - 1) \cdot \mathcal{V}_{\triangle}\left(\frac{1}{2^c}\frac{\log_2(s)}{c}\right)$$

However, this is not the only computation that affects the latency of scalar multiplication. The bucket aggregation step introduces additional overhead that cannot be ignored, as it involves extra bootstrap operations. In the original bucket method used in ECC, the bucket aggregation step is often optimized to minimize the total number of additions required. For instance, when the window size is 2 bits, the aggregation can be simplified from five additions to four as follows:

$$\begin{aligned} B_1 + 2B_2 + 3B_3 &= B_1 \overset{1}{+} B_2 \overset{2}{+} B_2 \overset{3}{+} B_3 \overset{4}{+} B_3 \overset{5}{+} B_3 \\ &= (B_1 \overset{2}{+} (B_2 \overset{1}{+} B_3)) \overset{3}{+} (B_2 \overset{1}{+} B_3) \overset{4}{+} B_3. \end{aligned} \tag{4}$$

Generally, using this method the total number of additions required in the bucket aggregation step is reduced to $2 \cdot (2^c - 2)$, which is beneficial as EC additions are the main bottleneck in this domain.

In contrast, in the TFHE case, the bottleneck is not the addition itself, but the number of CTB operations, which remains the same regardless of the optimization. Moreover, the

additions in this technique are sequential (first calculating $B_2+B_3$ before $B_1+B_2+B_3$), preventing the use of the optimized vector addition function. Therefore, it is more efficient to perform the aggregation naively by creating a new vector where each bucket result is added $i$ times, with $i$ the index of the bucket.

The optimized vector addition function only accepts ciphertexts where the carry bits are empty. To ensure this, we first perform a MC operation for each of the buckets, resulting in a total of $2^c-1$ MC operations (each requiring 2 CTB operations). Afterward, each bucket contains a message and a carry ciphertext that is added a specific number of times to the aggregation vector. This results in a total of $N=2\cdot\sum_{i=0}^{2^c-1} i=2^c(2^c-1)=2^{2c}-2^c$ elements in the vector. Since this vector consist of numerous bucket results (which are not blockshifted), the complexity of the vector addition operation will be $\mathcal{V}_{\square}(2^{2c}-2^c)$.

This means that the total number of bootstrap operations, for the precomputation, bucket accumulation and aggregation, in a single-scalar multiplication is:

$$nb_{CTB}=1+(2^c-1)\mathcal{V}_{\triangle}\left(\frac{1}{2^c}\frac{\log_2(s)}{c}\right)+2(2^c-1)+\mathcal{V}_{\square}\left(2^{2c}-2^c\right)$$

It is clear that increasing the window size reduces the number of CTB operations in the accumulation step but exponentially increases the operations in the aggregation step. In the next sections, we will focus on reducing the number of elements in the accumulation step as much as possible. Afterwards, we will try to reduce the overhead introduced by the aggregation step by introducing novel techniques that are specific to TFHE.

## 3.3 Sliding Window Scalar Multiplication

A known but less frequently used technique to reduce the number of elements in the accumulation buckets is the sliding window approach. The concept behind this technique is straightforward. Instead of pre-splitting the scalar into windows at fixed places, we consider the value of the scalar itself to determine the window placement. For example, if the least significant bit of a window is zero, we can shift that window with one position until the LSB is one, thus reducing the total number of windows.

More specifically, this technique starts at the least significant bit of the scalar and "slides" the window further by one or $c$ bits, depending on the bit value. In figure 4, a simple example using a 15-bit scalar with a three-bit window size is shown, illustrating how this technique reduces the total number of windows from five to four compared to the previous approach.

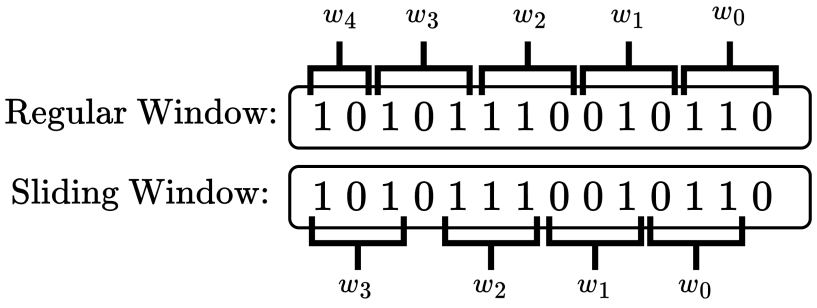



Figure 4: Example of how the sliding window technique reduces the total number of windows.

Generally, given a random distribution of bits in the scalar $s$ and a window size of $c$ bits, the average number of bits we pass each time we slide is $\frac{1}{2}(c+1)$. This is because there's a 1/2 chance of getting a 0, which causes the current window to slide one bit further. On the

other hand, there is a $1/2$ chance of getting a 1, which means this is a valid window that we process, after which we slide $c$ bits further to find the next window. Therefore, the estimated number of times we slide in the scalar $s$ is $\log_2(s)/(\frac{1}{2}(c+1))$. However, only half of these are valid windows (only when we slide $c$ bits). Hence, this technique reduces the total number of windows and, consequently, the total number of elements in the accumulation vector to:

$$N = \frac{2^c-1}{2^c} \cdot \log_2(s)/c \xrightarrow{\text{Sliding}} N = \frac{\log_2(s)}{c+1}.$$

One downside of this technique, and the reason it is less used in the EC field, is that the window placement is not known beforehand. This means we would need to precompute all multiples of two of the EC point, resulting in a significant increase in memory. However, in our TFHE case, this precomputation is not required as we can compute every multiplication of the ciphertext with a power of two at negligible cost.

Moreover, using this approach, only windows with an odd value will occur, thereby reducing the total number of buckets from $2^c-1$ to $2^{c-1}$. This reduction also decreases the overhead of the aggregation step as follows:

$$nb_{CTB,aggr} = 2(2^c-1) + \mathcal{V}_{\square}\left(2^{2c}-2^c\right) \xrightarrow{\text{Sliding}} 2(2^{c-1}) + \mathcal{V}_{\square}\left(2^{2c-1}\right)$$

The complexity can be derived using the fact that $2 \cdot \sum_{\substack{i=0 \\ i \text{ odd}}}^{2^c-1} i = 2 \cdot (2^{c-1})^2 = 2^{2c-1}$.

Combining this improvement in the number of CTB operations in the aggregation step with the reduced number of elements in the accumulation step, the total number of CTB operations for the scalar multiplication can be expressed as:

$$nb_{CTB} = 1 + 2^{c-1}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-1}}\frac{\log_2(s)}{c+1}\right) + 2(2^{c-1}) + \mathcal{V}_{\square}\left(2^{2c-1}\right)$$

## 3.4 Reducing Buckets

As discussed in the previous section, the bucket aggregation step introduces additional overhead that, depending on the window size and the scalar size, can outweigh the improvements made in the bucket accumulation stage. The primary factor affecting this aggregation overhead is the total number of required buckets. In this section, we will introduce an optimization that reduces the total number of buckets by half, with minimal computations, resulting in a significant decrease in aggregation overhead.

### 3.4.1 Negative Windows

The negative window technique is a well-known method in ECC that reduces the number of buckets by half with minimal cost [MO90]. The main idea is that, given a window size of $c$ bits, we rewrite the window values using signed windows, such that all window values become at most $c-1$ bits. Figure 5 illustrates how this technique modifies the scalar for a window size of four bits. We start with the least significant window, which initially holds the 4-bit value $(1001)_2 = 9$. In the normal bucket method, we would add the ciphertext corresponding with this scalar to $B_9$. However, this window value can also be expressed as $9 = 16-7 = 2^4-7$. Instead of adding the corresponding ciphertext to $B_9$, we add the negative of the ciphertext to $B_7$, while the $2^4$ term is carried over to the subsequent window. This process continues until all window values are reduced to three bits or less.

In general, the negative window technique modifies the window values from the original range of $[0 : 2^c-1]$ to $]-2^{c-1} : 2^{c-1}]$. This can be achieved by rewriting a window value of $2^{c-1}+a$ as $2^c-(2^{c-1}-a)$, where the new $2^c$ term is added as one to the next window, and the remaining $(2^{c-1}-a)$ term is at most $c-1$ bits.

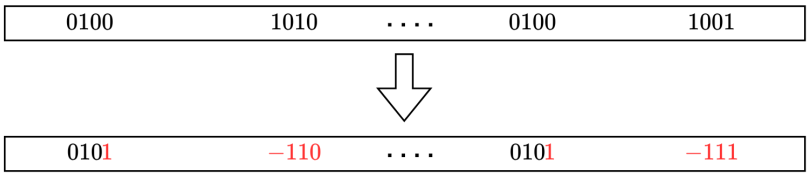



Figure 5: Simple Example of the Negative Windows technique

In ECC, this method reduces the number of buckets by half because the negative of an EC point can be computed efficiently with almost no extra cost. This results in a total of $2^{c-1}$ buckets, instead of $2^c-1$, while still using $c$-bit windows. This optimization can be combined with the sliding window technique, resulting in $2^{c-2}$ buckets for $c$-bit window sizes.

However, in the TFHE context, there is a drawback to this technique. While calculating the negative of an EC point is efficient, calculating the negative of a ciphertext in TFHE is not straightforward. Negating a ciphertext can cause an underflow, leading to a potential carry chain that requires a propagate function to clear the carry bits. This additional overhead removes the benefits of halving the number of buckets. In the next section, we will introduce a novel technique that avoids this overhead, allowing us to use the negative window optimization with minimal additional cost.

### 3.4.2 Negation with offset

In typical implementations of the negation function for TFHE, a carry propagation is required because the carry bits can be non-empty after the negation, leading to a significant number of CTB operations. To avoid this, we introduce a novel technique called *Negation with Offset*. This method creates an "offset" on the input, ensuring that the carry bits in the ciphertext remain empty after negation, thus eliminating the need for immediate carry propagation. The offset is removed at the end of the scalar multiplication, resulting in the correct solution.

For example, consider a large integer represented by multiple blocks of 2-bit message, 2-bit carry TFHE ciphertexts, where the carry bits are empty. To compute the negative of this ciphertext, we need to invert the sign on each individual block. The four possible options are:

$$0 \to -0 \quad 1 \to -1 \quad 2 \to -2 \quad 3 \to -3$$

To avoid negative value ciphertexts, which would require a carry propagation, we add an "offset" of three to each ciphertext block, making all possible options positive:

$$0 \to 3 \quad 1 \to 2 \quad 2 \to 1 \quad 3 \to 0.$$

This means that, for each ciphertext block $b_i$ we calculated $3-b_i$ instead of $-b_i$. Consequently, the carry bits of $3-b_i$ remain empty, so no carry propagation is needed before starting the bucket accumulation step of the scalar multiplication. This offset introduces an error on our result, but since we know the offset and the number of times a certain ciphertext is added to a bucket, we can compute the error and subtract it at the end of the algorithm. Note that taking the negative of the offset can be done efficiently as this is a plaintext integer.

This method can be generalized to any size ciphertext. For example, given a 3-bit window and the scalar $s=181$, the negation with offset technique works as follows:

First, in the preprocessing step, the ciphertext is multiplied by two, and the ciphertext negation with offset is calculated. In this example, we are working with the message modulus $2^m$. This means that calculating $3-b_i$ for each block $b_i$ in $ct$ is equivalent to calculating

$\overline{ct} = (2^m - 1) - ct$, which incurs no cost in terms of efficiency. Additionally, we need to calculate the negative of $ct_2$, denoted as $\overline{ct_2}$, which can also be done efficiently using a slightly different offset. Finally, on the scalar $s$, we apply the sliding negative window technique explained in the previous section.

$$\text{Preprocessing} \begin{cases} ct_2 = 2 \cdot ct \\ \overline{ct} = (2^m - 1) - ct \\ \overline{ct_2} = 2 \cdot ((2^m - 1) - ct) = (2^{m+1} - 2) - ct_2 \\ s = 181 = (10\,110\,101)_2 \xrightarrow{\text{Negative}} (11\,00\overline{1}\,0\overline{11})_2 \quad \text{with } \overline{1} = -1 \end{cases}$$

Note that the offset factor of $\overline{ct_2}$ differs because of the multiplication by two. Since we are using sliding negative windows, the total number of buckets is $2^{c-2}$. In our specific case, $c = 3$, which means we have $2^{c-2} = 2$ buckets, with indices one and three. Now we can begin with the first step of the bucket algorithm, the bucket accumulation.

The first window of the scalar $s$ is $(0\overline{11})_2 = -3$, so we add $\overline{ct}$ to bucket $B_3$. The second window is $(00\overline{1})_2 = -1$, requiring us to add $2^3 \cdot \overline{ct}$ to $B_1$. To calculate $2^3 \cdot \overline{ct}$, we rewrite it as $2^3 \cdot \overline{ct} = 2^2 \cdot \overline{ct_2} = \text{blockshift}(\overline{ct_2}, 1)$. The last window of $s$ is $(11)_2 = 3$, which means we add $2^6 \cdot ct = \text{blockshift}(ct, 3)$ to $B_3$.

$$\text{Bucket Accumulation} \begin{cases} B_1 & = \text{blockshift}(\overline{ct_2}, 1) \\ & = 4 \cdot ((2^{m+1} - 2) - ct_2) = 2^{m+3} - 8 - 4ct_2 \\ B_3 & = \overline{ct} + \text{blockshift}(ct, 3) \\ & = (2^m - 1) - ct + 64ct = 2^m - 1 + 63ct \end{cases}$$

After the accumulation, we obtain the correct result in each bucket with an additional offset. This offset will be compensated in the next step along with the bucket aggregation. The total offset added during the bucket accumulation step can be calculated as follows: The original offset is $2^m - 1$. We add $\overline{ct}$ to $B_3$, which means we added three times the original offset to the total result, giving $3(2^m - 1)$. Additionally, we added $2^3 \cdot \overline{ct}$ to $B_1$, which means we added an extra eight times the original offset. This results in a total offset, that we need to compensate, of $11(2^m - 1)$.

$$\begin{matrix} \text{Bucket Aggregation} \\ \text{Offset Compensation} \end{matrix} \begin{cases} s \cdot ct & = B_1 + 3B_3 - \text{offset} \\ & = (2^{m+3} - 8 - 4ct_2) + 3(2^m - 1 + 63ct) - 11 \cdot (2^m - 1) \\ & = 181ct \end{cases}$$

This technique is independent of the window size, the number of buckets, or the specific parameter set used. We only need a plaintext total offset counter to track how many times we add the negative of the ciphertext to the buckets. At the end, we can add this counter to bucket one before the vector addition begins, merging the offset compensation with the bucket accumulation step.

Furthermore, this novel offset technique is not limited to scalar multiplication but can also be applied to other TFHE applications where multiple subtractions are required to significantly reduce the total number of CTB operations.

With this novel approach, we can utilize the previously explained negative window technique to reduce the number of buckets by half, thereby significantly reducing the bucket aggregation overhead while introducing only one additional element in the vector addition (the total offset counter). Combining the negative window technique with the sliding window technique, the total number of buckets for a window size of $c$ bits becomes $2^{c-2}$. The aggregation complexity is reduced to:

$$nb_{CTB,aggr} = 2(2^{c-1}) + \mathcal{V}_{\square}\left(2^{2c-1}\right) \xrightarrow[\text{Windows}]{\text{Negative}} 2(2^{c-2}) + \mathcal{V}_{\square}\left(2^{2c-3} + 1\right)$$

This can be derived using $2\cdot\sum_{\substack{i=0\\ i \text{ odd}}}^{2^{c-1}} i = 2\cdot(2^{c-2})^2 = 2^{2c-3}$.

With this optimization, the total number of CTB operations is reduced to:

$$nb_{CTB} = 1+2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{\log_2(s)}{c+1}\right)+2(2^{c-2})+\mathcal{V}_{\square}\left(2^{2c-3}+1\right)$$

## 3.5 Bucket Aggregation Improvements

While the previous section halved the total number of buckets, the aggregation step still requires a vector addition with $2^{2c-3}$ elements, which still requires a substantial number of CTB operations for larger values of $c$. In this section, we present two novel optimizations that reduce the number of additions by leveraging a TFHE feature that is not available in the ECC case.

### 3.5.1 Powershift Aggregation

In the current bucket aggregation step, the total number of times a bucket is added to the aggregation vector depends on the bucket index, leading to a vector containing $2^{2c-3}+1 = \mathcal{O}(2^{2c})$ elements. However, in this section, we introduce a technique called powershift aggregation to reduce the number of elements to the order of $\mathcal{O}(2^c)$.

The idea behind powershift aggregation is that, by using the property that powers of four can be calculated for free in TFHE, we represent each bucket index using its power of four representation. With this representation, the number of times the bucket is added to the aggregation vector is minimized. For instance, the power of four representation of $55 = 4^3-2\cdot4-1$. This means that the total number of times $B_{55}$ is added to the aggregation vector is only four $(4^3B_{55}, -4B_{55}, -4B_{55}, -B_{55})$ instead of 55.

In general, using all previously explained optimizations, the possible bucket indices for a window size of $c$ bits are $1,3,5,\ldots,2^{c-1}-1$. We represent each of these indices using a series of powers of four as follows:

$$i = \sum_{k\geq 0} d_k 4^k \quad \text{with} \quad d_k \in \{-2,-1,0,1,2\}$$

The possible values for $d_i$ cannot be calculated efficiently, meaning that $d_i = 2$ implies adding the bucket twice to the aggregation vector. Consequently, the total number of times $B_i$ gets added to the aggregation vector can be expressed using following cost function:

$$N_{aggr,i} = 2\cdot\sum_{k\geq 0}|d_i| \quad \text{with} \quad d_i \in \{-2,-1,0,1,2\} \tag{5}$$

The times two is added because each bucket consists of both a message and carry ciphertext after the accumulation step.

This power of four representation is performed for each of the odd bucket indices from 1 until $2^{c-1}-1$. A closed formula giving the total number of elements for any window size of $c$ bits using this approach is hard to find. A recursive algorithm, given in Appendix B, was created to determine the most efficient representation for each of the indices that minimizes the total number of elements in the final aggregation vector, using the cost function defined in equation 5.

For $c \leq 6$, it is possible to derive a closed formula for the total number of elements, which equals:

$$N_{aggr} = \frac{5}{2}2^c - 4c^2 + 18c - 32.$$

This shows that, using this technique, the total number of elements is in the order of $\mathcal{O}(2^c)$ instead of $\mathcal{O}(2^{2c})$. Consequently, the total number of ciphertext bootstrap operations using the previous proposed optimizations is:

$$nb_{CTB} = 1 + 2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{\log_2(s)}{c+1}\right) + 2(2^{c-2}) + \mathcal{V}_{\square}\Big(N_{aggr}+1\Big) \quad \text{with } N_{aggr} = \mathcal{O}(2^c)$$

### 3.5.2 Bucket Merging

Although the PowerShift Aggregation optimization significantly reduces the overall aggregation overhead, the overhead remains substantial for larger window sizes ($c \geq 4$). In this section, we introduce a technique called Bucket Merging, which reduces the number of elements each bucket adds to the aggregation vector to a constant, independent of the bucket index.

Instead of representing the index of each bucket as a sum of powers of four, we can represent the index as the sum of another bucket index and a power of four. For instance, instead of representing 55 as $4^3 - 2 \cdot 4 - 1$, we can represent it as $4^3 - 9$, with 9 being another bucket index. This reduces the number of times $B_{55}$ is added to a vector from four to two. In general, it can be shown that all buckets are added to a vector exactly twice.

With this approach, we don't directly calculate the results of all buckets. Instead, we merge the bucket accumulation and aggregation steps. We begin with the higher-indexed buckets. After performing bucket accumulation on these buckets, we add them to another lower-indexed bucket. Then, these buckets start the accumulation process, and so on. For instance, with a five-bit window size, the Bucket Merging technique is illustrated in Figure 6 and operates as follows:

First, we perform the accumulation step for the orange buckets ($B_7$,$B_9$,$B_{11}$,$B_{13}$,$B_{15}$). Their results are then merged with the remaining buckets. For example, $B_7$ is added to bucket three, and $4B_7$ is added to bucket one (since $7 = 4+3$). After this process is completed for each bucket, we start the accumulation step for the purple buckets ($B_3$,$B_5$) and merge their results with $B_1$. Finally, $B_1$ starts its accumulation process to eventually achieve the final result of the scalar multiplication.

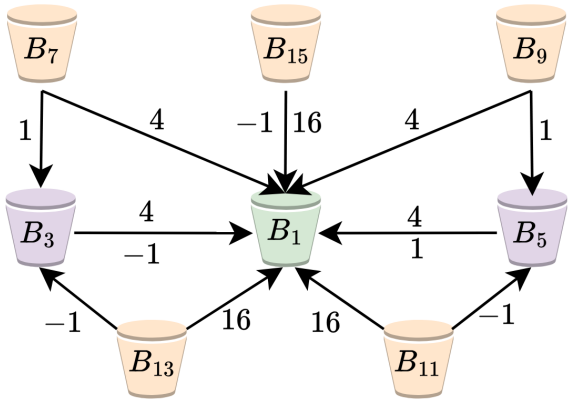



Figure 6: Visualization of the bucket merging technique for a window size of 5 bits

Using the Bucket Merging technique, the message and carry result for each bucket, except for $B_1$, are added to two other buckets. Consequently, the total number of elements generated during the aggregation stage is only four times the total number of buckets (without $B_1$) $: 4 \cdot (2^{c-2} - 1) = 2^c - 4$, which is an additional reduction compared to the PowerShift Aggregation technique for window sizes $c \geq 4$. For instance, when $c = 8$, the PowerShift Aggregation

technique adds approximately twice as many elements to the aggregation stage compared to the Bucket Merging technique.

With the bucket merging technique, the separate bucket aggregation step is eliminated, as it's integrated into the bucket accumulation process. To analyze the complexity, we can approximate the overhead from this merging technique by assuming that the aggregation step generates a vector with $2^c-4$ elements. However, instead of using the previously defined $\mathcal{V}_{\square}(x)$ function, we introduce a new function $\mathcal{V}'_{\square}(x)$ as follows: $\mathcal{V}'_{\square}(x)=\mathcal{V}_{\square}(x+5)=\frac{5}{8}x$. The $-5$ in the original $\mathcal{V}_{\square}(x)$ function accounts for the fact that a bootstrap is required only when there are more than five elements. In the bucket merging technique, the buckets are added to other accumulation vectors for which this $-5$ is already accounted for in the $\mathcal{V}_{\triangle}(x)$ complexity.

Therefore, the total number of CTB operations using all the previously explained optimizations is:

$$nb_{CTB}=1+2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{\log_2(s)}{c+1}\right)+2(2^{c-2}-1)+\mathcal{V}'_{\square}(2^c-4+1)$$

Figure 7 illustrates the results of our theoretical model compared to the measured complexity when implementing the complex formulas presented in the Zama TFHE-rs handbook [BdBB+25] for various window sizes. This comparison shows how the model we used throughout the paper and the assumptions made in the bucket merging technique are accurate.

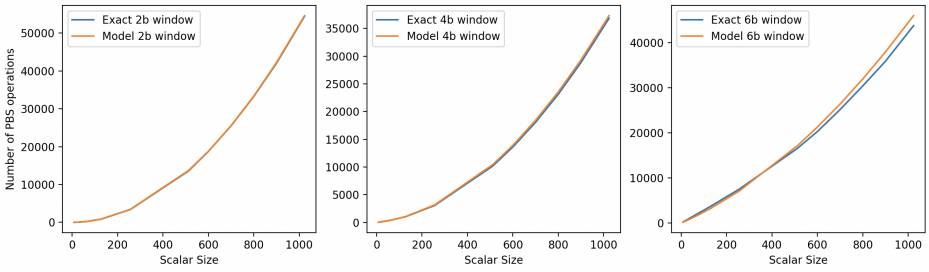



Figure 7: Comparison of the model used throughout the paper and the actual implementation for the total number of PBS operations using all proposed optimizations.

# 4 Multi-Scalar Multiplication

In the previous section, we adapted the well-known Pippenger method from the EC domain and applied it to the single-scalar multiplication in TFHE. However, the Pippenger method was originally designed for large multi-scalar multiplication problems. In this section, we will apply the TFHE-adapted Pippenger to the multi-scalar multiplication problem and demonstrate that we can use larger window sizes to achieve additional latency improvements. Furthermore, we will introduce a novel optimization to further reduce the number of CTB operations in the multi-scalar multiplication scenario.

## 4.1 Current Approach

Currently, there are no open-source implementations of a multi-scalar multiplication in TFHE, despite its relevance in numerous important applications, such as AI. To create a baseline, we start with the single-scalar multiplication and naively implement a vector-scalar

multiplication as a series of single-scalar multiplications followed by a final addition:

$$\begin{bmatrix} s_1 & s_2 & \cdots & s_k \end{bmatrix} \cdot \begin{bmatrix} ct_1 \\ ct_2 \\ \vdots \\ ct_k \end{bmatrix} = \sum_{i=1}^{k} s_i \cdot ct_i$$

Each scalar multiplication concludes with a full carry propagation to ensure that the carry bits are empty. We denote the number of CTB operations required for a full carry propagation as FP. In the preliminaries, we defined the number of CTB operations for this operation as 2, however, these are all sequential CTB operations that cannot be parallelized, so there exist optimizations that increase the total number of CTB operations while allowing for more parallelism. A detailed analysis on this operation can be found in the TFHE-rs handbook [BdBB+25].

For a vector of k elements, the total number of CTB operations required for the complete scalar multiplication is calculated as follows:

$$nb_{CTB} = k \cdot \underbrace{\left(1 + \mathcal{V}_{\triangle}\left(\frac{\log_2(s)}{2}\right) + \mathrm{FP}\right)}_{\text{One Scalar Multiplication}} + \mathcal{V}_{\square}(k)$$

We adopt this approach as the baseline for the multi-scalar multiplication.

## 4.2 Optimization on Current Approach

A minor optimization to the current approach is to replace the full propagate, that occurs before the final vector addition, with a message and carry extraction operation. With this approach, only a single MC operation (which requires 2 CTB operations that can be parallelized) needs to be performed on each scalar multiplication result, instead of a full propagate, before the final addition can start. The downside of this approach is that the final vector additions now consist of $2k$ elements, as each scalar multiplication result consists both of a message and a carry ciphertext.

$$nb_{CTB} = k \cdot \left(1 + \mathcal{V}_{\triangle}\left(\frac{\log_2(s)}{2}\right) + 2\right) + \mathcal{V}_{\square}(2k)$$

In the next section, we will improve on this approach by applying the previously introduced TFHE-adapted Pippenger method and introducing a novel optimization that minimizes the overall overhead.

## 4.3 Applying Single-Scalar TFHE-Pippenger to Multi-Scalar

The previously discussed bucket method was initially proposed to improve the complexity of multi-scalar multiplication problems within the EC domain. In this section, we demonstrate that our TFHE-adapted method can also be applied to these multi-scalar multiplication problems, resulting in substantial improvements compared to the current approach. Starting from equation 3, we adapt the bucket method for a vector of $k$ elements as follows:

$$\begin{bmatrix} s_1 & s_2 & \cdots & s_k \end{bmatrix} \cdot \begin{bmatrix} ct_1 \\ ct_2 \\ \vdots \\ ct_k \end{bmatrix} = \sum_{i=0}^{2^c-1} i \cdot B_i \quad \text{and} \quad B_i = \sum_{j=1}^{k} \sum_{\substack{w=0 \\ s_w == i}}^{W-1} ct_{j,w}.$$

This shows that instead of representing vector scalar multiplication as a series of individual scalar multiplications, we integrate the $k$ vector elements into the bucket accumulation step, while the aggregation step remains unchanged, regardless of the vector size. Logically, we can apply all the optimizations we previously explained for the bucket method to reduce the total number of CTB operations to:

$$nb_{CTB} = k + 2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{c+1}\right) + 2(2^{c-2}-1) + \mathcal{V}'_{\square}\left(2^c - 4 + 1\right)$$

Since the aggregation overhead remains constant regardless of the vector size, larger window sizes compared to the single-scalar multiplication case offer more advantages.

### 4.3.1 Bucket Doubling

One limitation in the multi-scalar multiplication approach is that for each ciphertext in the vector, we need to precompute $2 \cdot ct_i$, which can become a substantial number of CTB operations for a large $k$. In this section, we present a novel optimization called Bucket Doubling that eliminates the requirement for this precomputation.

In the bucket doubling method, we don't perform any precomputation at the start. Instead, we double the total number of buckets from $2^{c-2}$ to $2^{c-1}$. We define this additional set of buckets as $B'_i$. For instance, with a window size of three bits, we now have four buckets: $B_1, B_3, B'_1, B'_3$. The purpose of this extra set of buckets is that during the bucket accumulation step, when originally a precomputed $2 \cdot ct$ was added to bucket $B_i$, it is now added to bucket $B'_i$. After the accumulation step is complete, the bucket aggregation step begins for the $B'$ buckets. The result is afterward added twice to $B_1$, resulting in a total of four more elements in $B_1$ (first, a MC operation is performed to extract the message and carry). To get the final result, the bucket aggregation step starts for the $B$ bucket set. It's important to note that there's still only one correction factor because this is plaintext and can still be calculated regardless of the number of bucket sets.

In terms of complexity, we assume that the total number of elements in both bucket sets will be roughly the same. This results in the following number of CTB operations during the bucket accumulation step:

$$nb_{CTB,acc} = 2 \cdot 2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{2(c+1)}\right)$$

For the bucket aggregation complexity, we added an extra aggregation step for bucket set $B'$, which doesn't have a correction factor but introduces four extra elements into the $B$ bucket sets. Additionally, one MC operation is required on the result of bucket set $B'$:

$$nb_{CTB,aggr} = 2 \cdot 2(2^{c-2}-1) + \mathcal{V}'_{\square}\left(2^c - 4 + 1 + 4\right) + \mathcal{V}'_{\square}\left(2^c - 4\right) + 2$$

The total complexity for the bucket doubling method is then given by:

$$nb_{CTB} = 2 \cdot 2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{2(c+1)}\right) + 2 \cdot 2(2^{c-2}-1) + \mathcal{V}'_{\square}\left(2^c - 4 + 1 + 4\right) + \mathcal{V}'_{\square}\left(2^c - 4\right) + 2$$

The bucket doubling method isn't always advantageous. However, it becomes more beneficial when the number of vector elements, $k$, reaches a threshold. By comparing the theoretical complexities with and without bucket doubling, we can determine that $k$ must satisfy the following threshold:

$$k > \frac{47}{64}2^c$$

The derivation of the threshold is provided in Appendix A. Note that that this threshold is independent of the scalar size but only depends on the window size. Figure 8 illustrates the number of CTB operations required for the regular and bucket doubling methods for two distinct window sizes, clearly demonstrating that depending on the vector size and window size there is an almost additional $\times 2$ improvement that can be gained by using the bucket doubling method. We will use this threshold during our implementation to decide at runtime whether to employ the bucket doubling technique or not.

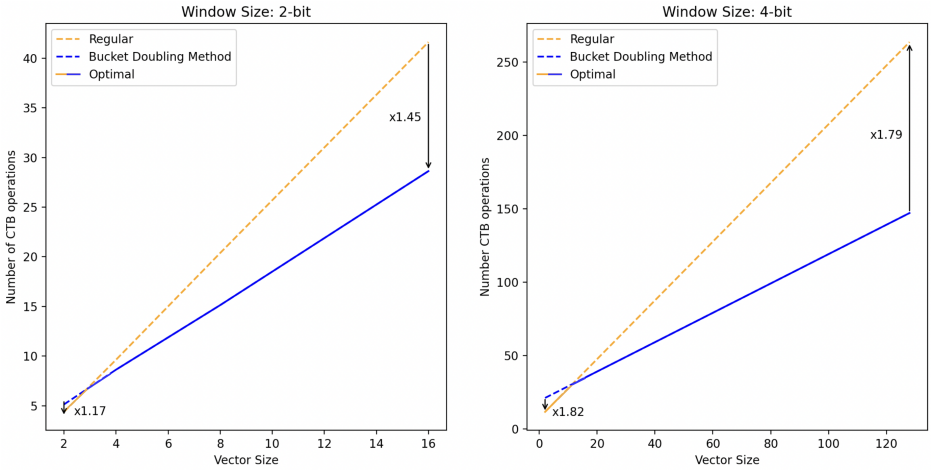



Figure 8: Comparison of the Number of CTB operations for the bucket doubling method for a 16-bit scalar and two different window sizes

# 5 Results

In this section, we will compare our implementation of single- and multi-scalar multiplication with the current state-of-the-art open-source implementations. For comparison with the Zama library, we used TFHE-rs v1.0.0 [Zam22], while for comparison with the work of Klemsa et al., we used Parmesan v0.1.5 [KO23]. All benchmarks were performed on the same Ubuntu 22.04 system equipped with an AMD EPYC 9174F 16-core processor running with 16 threads.

## 5.1 Single-Scalar Multiplication

In section 3, we presented several optimizations to minimize the number of PBS operations in a single-scalar multiplication. Figure 9 shows the impact of each optimization for a 64-bit and 256-bit scalar/ciphertext, while Table 3 gives an overview of the complexities for each optimization. Figure 9 also demonstrates that for larger scalar sizes, using a larger window size is advantageous. This is also illustrated in figure 10, which plots the single-scalar multiplication complexity for different window sizes and various scalar sizes. The reason behind this is that, for larger scalar sizes, more elements need to be stored and added together during the bucket accumulation step, while the bucket aggregation process remains independent of the scalar size. The window size essentially creates a trade-off between reducing the number of elements in the accumulation step and introducing a larger constant overhead in the aggregation step.

In Table 4, we present a comparison of our implementation of the single-scalar multiplication in TFHE with two existing open-source implementations. We ran the benchmarks for each implementation using 16 threads on a set of 200 random scalars.

Table 3: Comparison of the theoretical number of ciphertext bootstrap (CTB) operations for each of the proposed optimizations for a single-scalar multiplication of the scalar $s$ with a TFHE ciphertext. $c$ indicates the window size.

| | Precomp | Accumulation | Aggregation |
|---|---|---|---|
| TFHE-rs [Zam22] | 1 | $\mathcal{V}_{\triangle}\left(\frac{\log_2(s)}{2}\right)$ | 0 |
| TFHE Pippenger | 1 | $(2^c-1)\mathcal{V}_{\triangle}\left(\frac{1}{2^c}\frac{\log_2(s)}{c}\right)$ | $2(2^c-1)+\mathcal{V}_{\square}\left(2^{2c}-2^c\right)$ |
| Sliding Windows | 1 | $2^{c-1}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-1}}\frac{\log_2(s)}{c+1}\right)$ | $2(2^{c-1})+\mathcal{V}_{\square}\left(2^{2c-1}\right)$ |
| Negative Windows | 1 | | $2(2^{c-2})+\mathcal{V}_{\square}\left(2^{2c-3}+1\right)$ |
| Powershift Aggregation | 1 | | $2(2^{c-2})+\mathcal{V}_{\square}\left(\mathcal{O}(2^c)+1\right)$ |
| Bucket Merging | 1 | | $2(2^{c-2}-1)+\mathcal{V}'_{\square}(2^c-4+1)$ |

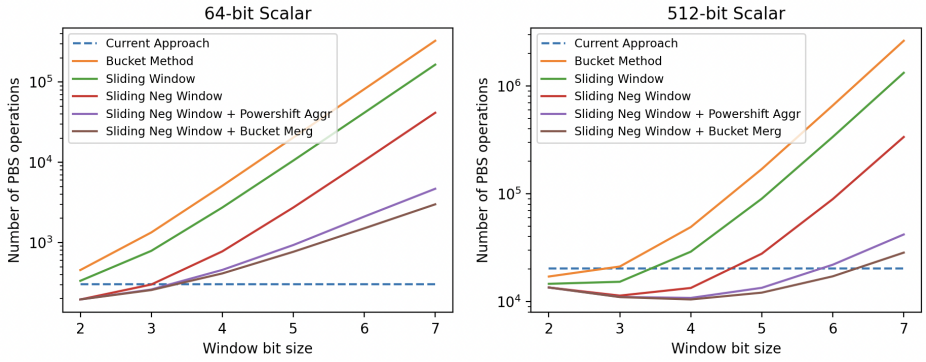



Figure 9: Comparison of number of PBS operations for all proposed optimizations for two different scalar sizes

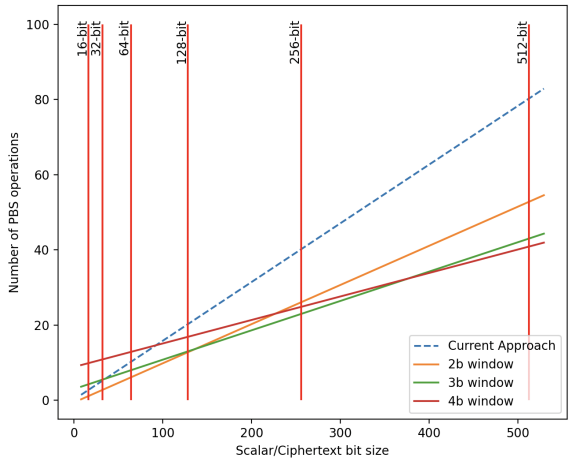



Figure 10: Number of PBS operations for different window sizes

The implementation in the TFHE-rs library was explained in section 3.1. In the original implementation, the scalar multiplication ends with a complete carry propagation to clear the carry bits for the next computation. To more clearly illustrate our improvements, we disabled this operation for all implementations in Table 4. The improvement column in Table 4 compares our implementation to the implementation of TFHE-rs. Depending on the size of the scalar and ciphertext, we achieve a speedup of up to ×2.05. The table also shows that for larger scalar sizes, it is more advantageous to use larger window sizes.

Table 4: Latency for a single-scalar multiplication without final full propagation. Using the 2-bit message, 2-bit carry parameter set with 128-bit security and 16 threads.

| Scalar/Ctx Size | Parmesan [KO23] | TFHE-rs [Zam22] | Ours [Window Size] | | | Improv over TFHE-rs |
|---|---|---|---|---|---|---|
| | | | [2b] | [3b] | [4b] | |
| 8/8-bit | 88ms | 30ms | **22ms** | 65ms | 109ms | ×1.38 |
| 16/16-bit | 209ms | 49ms | **34ms** | 98ms | 160ms | ×1.44 |
| 32/32-bit | 657ms | 95ms | **68ms** | 113ms | 246ms | ×1.39 |
| 64/64-bit | / | 343ms | **235ms** | 275ms | 485ms | ×1.46 |
| 128/128-bit | / | 1.357s | 920ms | **846ms** | 1.119s | ×1.60 |
| 256/256-bit | / | 5.492s | 3.641s | **3.041s** | 3.273s | ×1.81 |
| 512/512-bit | / | 21.973s | 14.500s | 11.752s | **10.721s** | ×2.05 |

## 5.2 Multi-Scalar Multiplication

Table 5 provides an overview of the complexities associated with each optimization, for the multi-scalar multiplication, discussed in this paper. Table 6 presents the results for multi-scalar multiplication across various vector and scalar sizes, comparing them to the current approach described in Section 4.1. The "ours 1b" approach is the minor optimization that we discussed in section 4.2. More detailed results can be found in Appendix C. Note that even larger window sizes give improved performance compared to the single-scalar multiplication scenario. This is because in the multi-scalar multiplication case, the number of CTB operations in the accumulation step is influenced by both the scalar bit size and the size of the vector, while the bucket aggregation step is independent of both. For example, for a vector of 1024 elements of 16 bits, we achieve an improvement up to ×5.97 compared to the current approach.

To demonstrate the importance of accelerating scalar multiplication problems in specific FHE AI applications, we used the AlexNet architecture as an example. AlexNet [KSH17] is one of the most widely recognized image recognition AI architectures. Within this architecture, all linear layers can be broken down into multi-scalar multiplication problems. Some of these layers are particularly large, especially the last two fully connected layers. In Table 7, we compare the latency of performing one multi-scalar multiplication with the kernel sizes used in the AlexNet architecture, showing that we achieve an improvement up to ×7.51 compared to the current approach.

Table 5: Comparison of the theoretical number of ciphertext bootstrap (CTB) operations for each of the proposed optimizations for a multi-scalar multiplication of $k$ scalars and TFHE ciphertexts. $c$ indicates the window size.

| | Precomp | Accumulation | Aggregation |
|---|---|---|---|
| TFHE-rs [Zam22] | $k$ | $k \cdot \left( \mathcal{V}_{\triangle} \left( \frac{\log_2(s)}{2} \right) + \text{FP} \right)$ | $\mathcal{V}_{\square}(k)$ |
| Prevent Carry Propagate | $k$ | $k \cdot \left( \mathcal{V}_{\triangle} \left( \frac{\log_2(s)}{2} \right) + 2 \right)$ | $\mathcal{V}_{\square}(2k)$ |
| TFHE Pippenger | $k$ | $2^{c-2} \mathcal{V}_{\triangle} \left( \frac{1}{2^{c-2}} \frac{k \log_2(s)}{c+1} \right)$ | $2(2^{c-2}-1) + \mathcal{V}'_{\square}(2^c - 4 + 1)$ |
| Bucket Doubling | 0 | $2 \cdot 2^{c-2} \mathcal{V}_{\triangle} \left( \frac{1}{2^{c-2}} \frac{k \log_2(s)}{2(c+1)} \right)$ | $2 \cdot 2(2^{c-2}-1) + \mathcal{V}'_{\square}(2^c - 4) + \mathcal{V}'_{\square}(2^c - 4 + 1 + 4) + 2$ |

Table 6: Latency for a multi-scalar multiplication without final full propagation. Using the 2-bit message, 2-bit carry parameter set with 128-bit security and 16 threads. b and e respectively indicate the bit size of the scalar and the number of elements in the vector.

| Scalar / Vector | TFHE-rs [Zam22] | Ours 1b | Window Size | Latency | Improv over TFHE-rs |
|---|---|---|---|---|---|
| 8b / 8e | 205ms | 176ms | 2 | 81ms | ×2.53 |
| 8b / 128e | 1.879s | 2.075s | 2 | 545ms* | ×3.45 |
| 8b / 1024e | 14.280s | 16.053s | 6 | 2.717s* | ×5.26 |
| 16b / 8e | 480ms | 338ms | 2 | 197ms | ×2.44 |
| 16b / 128e | 6.247s | 5.069s | 4 | 1.550s* | ×4.03 |
| 16b / 1024e | 49.218s | 40.260s | 7 | 8.242s* | ×5.97 |
| 32b / 8e | 1.258s | 977ms | 2 | 544ms* | ×2.31 |
| 32b / 128e | 18.968s | 15.193s | 5 | 4.774s* | ×3.97 |
| 32b / 1024e | 149.467s | 120.388s | 8 | 27.743s* | ×5.39 |
| 64b / 8e | 3.839s | 3.190s | 4 | 1.674s | ×2.29 |
| 64b / 128e | 59.940s | 50.343s | 6 | 16.108s* | ×3.72 |
| 64b / 1024e | 477.218s | 407.301s | 8 | 94.064s* | ×5.07 |

*With Bucket Doubling Technique

Table 7: Latency for a multi-scalar multiplication without final full propagation for specific sizes used in the AlexNet AI Architecture, utilizing a 16-bit scalar and ciphertext. We employ a 2-bit message, a 2-bit carry parameter set, 128-bit security, and 16 threads.

| Layer | Kernel Size | TFHE-rs [Zam22] | Window Size | Latency | Improv over TFHE-rs |
|---|---|---|---|---|---|
| $CONV_2$ | $5\times5\times96=2400$ | 115.27s | 7 | 17.512s* | $\times$6.58 |
| $FC_1$ | $1\times1\times9216=9216$ | 444.06s | 9 | 59.092s* | $\times$7.51 |
| $FC_2$ | $1\times1\times4096=4096$ | 199.13s | 8 | 28.219s* | $\times$7.06 |

*With Bucket Doubling Technique

# 6 Conclusion

The multi-scalar multiplication is a critical operation in applications such as privacy-preserving AI, making it essential to optimize. In this paper, we demonstrated that this operation can be accelerated significantly by taking inspiration from the bucket method, well-known in the elliptic curve domain but not applied before in FHE. We adapted the bucket method to leverage the unique features of TFHE and introduced novel optimizations to make further improvements. Additionally, we found that the bucket method improves performance in both single- and multi-scalar multiplication scenarios, despite its original design for only large multi-scalar multiplication cases. Our results show that depending on the scalar size, single-scalar multiplication can achieve up to a $\times$2.05 speedup. For multi-scalar multiplication, we achieve improvements up to $\times$7.51.

**Acknowledgments** This work was supported in part by the Horizon 2020 ERC Advanced Grant (101020005 Belfort) and the CyberSecurity Research Flanders with reference number VOEWICS02


## References

[BdBB+25] Mathieu Ballandras, Mayeul de Bellabre, Loris Bergerat, Charlotte Bonte, Carl Bootland, Benjamin R. Curtis, Jad Khatib, Jakub Klemsa, Arthur Meyre, Thomas Montaigu, Jean-Baptiste Orfila, Nicolas Sarlin, Samuel Tap, and David Testé. Tfhe-rs: A (practical) handbook. https://github.com/zama-ai/tfhe-rs-handbook/blob/main/tfhe-rs-handbook.pdf, 2025.

[BGMW92] Ernest Brickell, Daniel Gordon, Kevin McCurley, and David Wilson. Fast exponentiation with precomputation (extended abstract). pages 200–207, 01 1992.

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, page 309–325, New York, NY, USA, 2012. Association for Computing Machinery.

[CGGI18] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. Cryptology ePrint Archive, Paper 2018/421, 2018.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017*, pages 409–437, Cham, 2017. Springer International Publishing.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption, 2012.

[KO23] Jakub Klemsa and Melek Onen. PARMESAN: Parallel ARithMEticS over ENcrypted data. Cryptology ePrint Archive, Paper 2023/544, 2023.

[KSH17] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Commun. ACM*, 60(6):84–90, May 2017.

[MO90] François Morain and Jorge Olivos. Speeding up the computations on an elliptic curve using addition-subtraction chains. *ITA*, 24:531–544, 01 1990.

[Pip76] Nicholas Pippenger. On the evaluation of powers and related problems (preliminary version). In *17th Annual Symposium on Foundations of Computer Science, Houston, Texas, USA, 25-27 October 1976*, pages 258–263. IEEE Computer Society, 1976.

[Zam22] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. https://github.com/zama-ai/tfhe-rs.

# Appendices

## A Derivation of Minimum Number of Vector elements for Bucket Doubling Optimization

We compare the number of CTB operations using the bucket doubling optimization with the one without, and determine the point at which the bucket doubling method complexity becomes lower.

$$
\begin{aligned}
0 \geq 2+2\cdot 2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{2(c+1)}\right)+2\cdot 2(2^{c-2}-1)+\mathcal{V}'_{\square}\left(2^c-4+1+4\right)+\mathcal{V}'_{\square}\left(2^c-4\right) \\
-\left(k+2^{c-2}\mathcal{V}_{\triangle}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{c+1}\right)+2(2^{c-2}-1)+\mathcal{V}'_{\square}\left(2^c-4+1\right)\right) \\
k \geq 2+2\cdot 2^{c-2}\frac{5}{16}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{2(c+1)}-5\right)+2(2^{c-2}-1)+\mathcal{V}'_{\square}\left(2^c\right)-2^{c-2}\frac{5}{16}\left(\frac{1}{2^{c-2}}\frac{k\log_2(s)}{c+1}-5\right) \\
k \geq 2-\frac{50}{16}\cdot 2^{c-2}+2(2^{c-2}-1)+\frac{5}{8}2^c+\frac{25}{16}\cdot 2^{c-2} \\
k \geq \frac{47}{64}2^c
\end{aligned}
$$

## B Powershift Aggregation Algorithm

**Algorithm 1** Powershift Aggregation Simulator

```
function TOTALCOST(c)
    total = 0
    for i ← 1 to 2^{c-1} with 2 do
        total += 2·MINCOST(i, ⌊c/2⌋)        ▷ 2· because message and carry
    end for
    return total
end function

function MINCOST(i, k)
    if k < 0 then
        return 0 if i == 0 else INF
    end if
    best = INF
    for d in {−2,−1,0,1,2} do
        remain = i − d·4^k
        cost = |d| + MINCOST(remain, k − 1)
        if cost < best then
            best ← cost
        end if
    end for
    return best
end function
```

## C Detailed Multi-scalar Multiplication Results

Table 8: Latency for a multi-scalar multiplication without final full propagation. Using the 2-bit message, 2-bit carry parameter set with 128-bit security and 16 threads.

| Scalar/Vector | TFHE-rs [Zam22] | Ours 1b | Ours 2b | Ours 3b | Ours 4b | Ours 5b | Ours 6b | Ours 7b | Ours 8b | Ours 9b | Ours 10b | Improv |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 8b / 8e | 205ms | 176ms | **81ms** | 124ms | 175ms | 256ms | 347ms | 371ms | 426ms | 543ms | 544ms | ×2.53 |
| 8b / 16e | 300ms | 303ms | **145ms** | 172ms | 203ms | 294ms | 390ms | 447ms | 516ms | 622ms | 656ms | ×2.07 |
| 8b / 32e | 545ms | 565ms | **235ms*** | 277ms | 315ms | 364ms | 468ms | 582ms | 713ms | 865ms | 906ms | ×2.32 |
| 8b / 64e | 993ms | 1.066s | **330ms*** | 436ms* | 520ms | 543ms | 616ms | 787ms | 996ms | 1.194s | 1.247s | ×3.01 |
| 8b / 128e | 1.879s | 2.075s | **545ms*** | 568ms* | 660ms* | 751ms* | 850ms* | 1.098s | 1.406s | 1.713s | 1.792s | ×3.45 |
| 8b / 256e | 3.634s | 3.995s | 991ms* | **941ms*** | 960ms* | 985ms* | 1.050s* | 1.311s* | 1.894s* | 2.379s* | 2.496s* | ×3.86 |
| 8b / 512e | 7.165s | 7.987s | 1.791s* | 1.625s* | **1.581s*** | 1.629s* | 1.591s* | 1.769s* | 2.291s* | 2.945s* | 3.159s* | ×4.53 |
| 8b / 1024e | 14.280s | 16.053s | 3.576s* | 3.099s* | 2.777s* | 2.815s* | **2.717s*** | 2.774s* | 3.050s* | 3.765s* | 4.308s* | ×5.26 |
| 16b / 8e | 480ms | 338ms | **197ms** | 205ms | 231ms | 360ms | 526ms | 680ms | 883ms | 1.108s | 1.363s | ×2.44 |
| 16b / 16e | 873ms | 649ms | **313ms*** | 348ms* | 385ms | 459ms | 659ms | 926ms | 1.253s | 1.570s | 1.992s | ×2.79 |
| 16b / 32e | 1.685s | 1.298s | 543ms* | **539ms*** | 585ms* | 707ms* | 867ms | 1.228s | 1.689s | 2.225s | 2.855s | ×3.13 |
| 16b / 64e | 3.253s | 2.512s | 980ms* | 890ms* | **886ms*** | 976ms* | 1.258s* | 1.592s | 2.257s | 3.092s | 4.073s | ×3.67 |
| 16b / 128e | 6.247s | 5.069s | 1.888s* | 1.624s* | **1.550s*** | 1.574s* | 1.717s* | 2.258s* | 3.089s | 4.300s | 5.888s | ×4.03 |
| 16b / 256e | 12.348s | 10.046s | 3.660s* | 3.080s* | 2.779s* | **2.699s*** | 2.738s* | 3.064s* | 4.352s* | 6.089s | 8.454s | ×4.58 |
| 16b / 512e | 25.007s | 20.001s | 7.379s* | 6.059s* | 5.161s* | 4.807s* | **4.591s*** | 4.802s* | 5.618s* | 8.339s* | 11.975s | ×5.45 |
| 16b / 1024e | 49.218s | 40.260s | 14.652s* | 11.710s* | 9.842s* | 8.881s* | 8.451s* | **8.242s*** | 8.745s* | 10.796s* | 15.689s* | ×5.97 |

*With Bucket Doubling Technique

Table 9: Latency for a multi-scalar multiplication without final full propagation. Using the 2-bit message, 2-bit carry parameter set with 128-bit security and 16 threads.

| Scalar/Vector | TFHE-rs [Zam22] | Ours 1b | Ours 2b | Ours 3b | Ours 4b | Ours 5b | Ours 6b | Ours 7b | Ours 8b | Ours 9b | Ours 10b | Improv |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 32b / 8e | 1.258s | 977ms | **544ms*** | 555ms | 586ms | 723ms | 1.135s | 1.736s | 2.441s | 3.302s | 4.265s | ×2.31 |
| 32b / 16e | 2.551s | 1.938s | 990ms* | **918ms*** | 978ms* | 1.090s | 1.394s | 2.181s | 3.259s | 4.535s | 6.009s | ×2.78 |
| 32b / 32e | 4.902s | 3.867s | 1.880s* | 1.623s* | **1.586s*** | 1.775s* | 2.078s | 2.721s | 4.185s | 6.208s | 8.486s | ×3.09 |
| 32b / 64e | 9.651s | 7.592s | 3.674s* | 3.011s* | **2.769s*** | 2.800s* | 3.161s* | 3.837s | 5.271s | 8.066s | 11.552s | ×3.49 |
| 32b / 128e | 18.968s | 15.193s | 7.169s* | 5.748s* | 5.044s* | **4.774s*** | 4.919s* | 5.726s* | 7.328s | 10.085s | 15.441s | ×3.97 |
| 32b / 256e | 37.574s | 30.424s | 14.681s* | 11.625s* | 9.508s* | 8.606s* | **8.379s*** | 8.914s* | 10.735s* | 14.062s | 20.117s | ×4.48 |
| 32b / 512e | 74.846s | 60.897s | 29.527s* | 23.058s* | 18.935s* | 16.469s* | 15.253s* | **15.202s*** | 16.571s* | 20.520s* | 27.130s | ×4.92 |
| 32b / 1024e | 149.467s | 120.388s | 58.337s* | 45.842s* | 37.771s* | 32.082s* | 29.015s* | 27.834s* | **27.743s*** | 30.822s* | 39.206s* | ×5.39 |
| 64b / 8e | 3.839s | 3.190s | 1.945s* | 1.748s* | **1.674s** | 1.816s | 2.448s | 3.952s | 6.119s | 8.827s | 12.085s | ×2.29 |
| 64b / 16e | 7.701s | 6.429s | 3.699s* | 3.118s* | **2.990s*** | 3.045s | 3.478s | 4.823s | 7.815s | 11.785s | 16.452s | ×2.58 |
| 64b / 32e | 15.194s | 12.898s | 7.310s* | 5.854s* | **5.246s*** | 5.231s* | 5.611s | 6.550s | 9.341s | 15.053s | 22.619s | ×2.9 |
| 64b / 64e | 30.154s | 25.428s | 14.473s* | 11.344s* | 9.712s* | **8.970s*** | 9.325s* | 10.380s | 12.341s | 18.427s | 29.340s | ×3.36 |
| 64b / 128e | 59.940s | 50.343s | 28.309s* | 22.663s* | 18.474s* | 16.549s* | **16.108s*** | 17.137s* | 19.555s | 23.698s | 35.103s | ×3.72 |
| 64b / 256e | 119.385s | 101.332s | 58.955s* | 45.494s* | 36.949s* | 31.644s* | 29.059s* | **28.656s*** | 31.501s* | 37.537s | 46.557s | ×4.17 |
| 64b / 512e | 239.842s | 204.799s | 120.031s* | 92.610s* | 72.764s* | 62.814s* | 55.804s* | **52.321s*** | 53.004s* | 59.356s* | 71.323s | ×4.58 |
| 64b / 1024e | 477.218s | 407.301s | 239.875s* | 181.718s* | 147.467s* | 124.318s* | 108.545s* | 98.888s* | **94.064s*** | 96.666s* | 109.768s* | ×5.07 |

*With Bucket Doubling Technique